

PROYECTO: TAD Postfija

Profesor(a): Ariel Adara Mercado Martinez

Asignatura: Estructura de Datos y Algoritmos

Grupo: 5

Integrante(s): Gonzalez Pérez Luis Angel

Esquivel Cortés Iker Alonso

Batalla Ramírez Galileo

Equipo: Brigada 8

Semestre: SEMESTRE 2026-2

Fecha de entrega: 17/05/2026

Observaciones:

CALIFICACIÓN: _____

Estructura de Datos y Algoritmos

PROYECTO: TAD Postfija.

INTRODUCCIÓN:

En matemáticas y programación, las expresiones aritméticas pueden escribirse en diferentes notaciones. La notación infija es la más común para las personas, ya que coloca los operadores entre los operandos, por ejemplo:

$$(a+b) \cdot c$$

Sin embargo, esta notación requiere considerar reglas como la precedencia de operadores y el uso de paréntesis para determinar el orden correcto de evaluación.

Una alternativa es la notación postfija, también conocida como notación polaca inversa, en la cual los operadores se colocan después de los operandos. Esta representación facilita el procesamiento automático de expresiones porque elimina la necesidad de paréntesis y simplifica la evaluación mediante estructuras tipo pila.

El objetivo del proyecto fue implementar un programa en lenguaje C capaz de:

- Leer variables y expresiones desde un archivo de texto.
- Convertir expresiones de notación infija a postfija.
- Evaluar la expresión postfija obtenida.
- Utilizar exclusivamente estructuras dinámicas de datos como pilas y colas.

EXPLICACIÓN DE ESTRUCTURAS UTILIZADAS:

Implementación de pila:

La pila fue implementada mediante listas enlazadas dinámicas. Cada nodo almacena un dato y un apuntador al siguiente nodo.

La estructura utilizada fue:

```
6  typedef struct NodoPila {
7      void *dato;
8      struct NodoPila* siguiente;
9  } NodoPila;
10
11 typedef struct {
12     NodoPila* tope;
13 } Pila;
```

La pila funciona bajo el principio LIFO (Last In First Out), donde el último elemento insertado es el primero en salir.

Las operaciones principales implementadas fueron:

- push() : inserta elementos.
- pop() : elimina el elemento del tope.
- peek() : consulta el elemento superior.
- pilaVacía() : verifica si la pila contiene elementos.

La pila se utilizó principalmente para:

- Manejar operadores durante la conversión infija a postfija.
- Almacenar operandos durante la evaluación de expresiones.

Implementación de cola:

La cola también fue implementada mediante listas enlazadas dinámicas, la estructura utilizada fue:

```

6  typedef struct NodoCola {
7      void *dato;
8      struct NodoCola* siguiente;
9  } NodoCola;
10
11  typedef struct {
12      NodoCola* frente;
13      NodoCola* final;
14  } Cola;
15

```

La cola trabaja bajo el principio FIFO (First In First Out), donde el primer elemento insertado es el primero en salir.

Las operaciones principales fueron:

- enqueue() : inserta elementos al final.
- dequeue() : elimina elementos del frente.
- colaVacía() : verifica si la cola está vacía.

La cola fue utilizada para almacenar la expresión en notación postfija.

Uso de memoria dinámica:

Todo el proyecto fue desarrollado utilizando memoria dinámica mediante malloc() y free().

Cada nodo de pila y cola se reserva dinámicamente en memoria durante la ejecución del programa. Esto permite:

- Evitar límites fijos de tamaño.
- Administrar memoria según las necesidades reales.
- Cumplir con la restricción de no usar arreglos para simular pilas o colas.

Además, cada estructura libera correctamente la memoria utilizada al finalizar mediante funciones de destrucción.

ALGORITMO DE CONVERSIÓN:

Precedencia:

La precedencia determina qué operadores deben evaluarse primero, en el programa se manejó la siguiente jerarquía:

1. Paréntesis ($()$)
2. Potencia ($^$)
3. Multiplicación y división ($*$, $/$)
4. Suma y resta ($+$, $-$)

Por ejemplo: $a+b \cdot c$

Primero realiza la multiplicación y después la suma.

Asociatividad:

La asociatividad determina el orden de evaluación cuando existen operadores con la misma precedencia.

Por ejemplo: $a-b-c$

Se evalúa de izquierda a derecha.

El algoritmo considera estas reglas al decidir cuándo desapilar operadores.

Funcionamiento del algoritmo infija $\rightarrow$ postfija:

Para convertir la expresión se implementó el algoritmo conocido como *Shunting Yard*.

El procedimiento general es:

1. Leer la expresión carácter por carácter.
2. Si el carácter es un operando:
 - Se envía directamente a la cola de salida.
3. Si es un operador:

- Se comparan precedencias con los operadores almacenados en la pila.
 - Dependiendo del resultado, se desapilan operadores hacia la cola.
4. Si aparece un paréntesis:
 - Se controla el agrupamiento correspondiente.
 5. Al finalizar:
 - Todos los operadores restantes pasan a la cola.

Por ejemplo, la expresión: $(a+b) \cdot c$

se transforma en: $ab+c^*$

Evaluación postfija:

La evaluación de la expresión postfija se realizó utilizando una pila de operandos.

El procedimiento consiste en:

1. Leer cada símbolo de la expresión postfija.
2. Si es un operando:
 - Se inserta en la pila.
3. Si es un operador:
 - Se extraen los dos operandos superiores.
 - Se realiza la operación correspondiente.
 - El resultado vuelve a insertarse en la pila.
4. Al finalizar:
 - El único elemento restante en la pila corresponde al resultado final.

Por ejemplo, para: $ab+c^*$, con: $a=5$, $b=3$, $c=2$, el resultado obtenido es:

$$(5+3) \cdot 2=16$$

Casos de prueba:

Caso 1:

```
TAD_postfija > input > caso1.txt
1  a = 5
2  b = 3
3  c = 2
4
5  expr=(a+b)*c

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
powershell + - [ ] [ ] ... | [ ] [ ] X

PS C:\Users\ikera\OneDrive\Desktop\EDA\TAD_postfija> gcc main.c cola.c pila.c parser.c evaluator.c -o programa
PS C:\Users\ikera\OneDrive\Desktop\EDA\TAD_postfija> ./programa
Expresion infija:
(a+b)*c

Expresion postfija:
a b + c *

Resultado:
16.00
```

Caso 2:

```
TAD_postfija > input > caso2.txt
1  a = 5
2  b = 1
3  c = 2
4  expr=(a+b)*c

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
powershell + - [] [] ... | [] x

PS C:\Users\ikera\OneDrive\Desktop\EDA\TAD_postfija> gcc main.c cola.c pila.c parser.c evaluator.c -o programa
PS C:\Users\ikera\OneDrive\Desktop\EDA\TAD_postfija> ./programa
Expresion infija:
(a+b)*c

Expresion postfija:
a b + c *

Resultado:
12.00
```

Caso 3:

```
TAD_postfija > input > caso3.txt
1  a = 4
2  b = 6
3  c = 2
4  (a+b)*c

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
powershell + - [] [] ... | [] x

PS C:\Users\ikera\OneDrive\Desktop\EDA\TAD_postfija> gcc main.c cola.c pila.c parser2.c evaluator2.c -o programa
PS C:\Users\ikera\OneDrive\Desktop\EDA\TAD_postfija> ./programa
Expresion infija:
(a+b)*c

Expresion postfija:
a b + c *

Resultado:
20.00
```

Conclusión del Equipo:

Durante el desarrollo del proyecto se presentaron diversas dificultades relacionadas con:

- El manejo manual de memoria dinámica.
- La implementación correcta de pilas y colas enlazadas.
- La conversión adecuada entre notación infija y postfija.
- El control de precedencia y paréntesis.

Una de las decisiones principales de diseño fue utilizar estructuras dinámicas mediante listas enlazadas en lugar de arreglos, con el fin de cumplir las restricciones establecidas y permitir una implementación más flexible.

El proyecto nos permitió comprender mejor:

- El funcionamiento interno de las expresiones aritméticas.
- El uso práctico de estructuras dinámicas.

- La administración manual de memoria en C.
- Y la aplicación de algoritmos clásicos de parsing y evaluación de expresiones.

REFERENCIAS:

- **González, G. I. C. (s. f.). *Lic. en Informática*.**
https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/1453/mod_resource/content/1/contenido/index.html
- ***Evaluar expresión postfija. (2023, 21 marzo). El Profe Ariel.***
<https://elprofearieloficial.wordpress.com/estructuras-de-datos/evaluar-expresion-postfija/>